

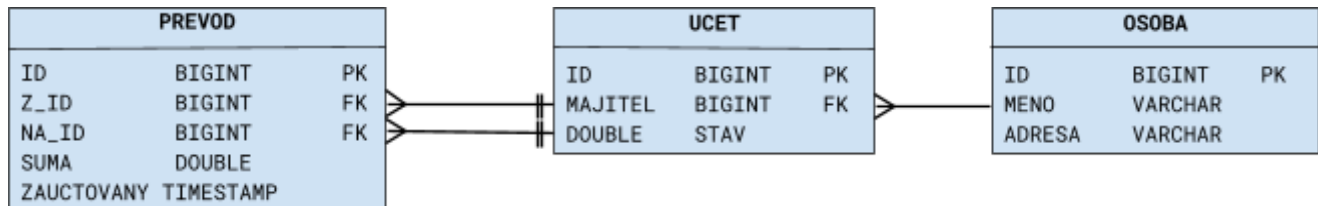
V zip súbore, ktorý ste si stiahli sa nachádzajú aj adresáre **dbapp** a **restapp**.

Implementačná úloha 1.

dbapp je maven-projekt, pre implementáciu **databázovej aplikácie**. Projekt už obsahuje:

- súbor **pom.xml** so všetkými závislosťami pre implementáciu zadania, nie je ho potrebné modifikovať.
- súbor **persistence.xml** s konfiguračnými údajmi pre databázu postgres (beží vo virtuálnom stroji).
Poznámka. Postgres je referenčná databáza pre testovanie a hodnotenie implementácie.
Pri vývoji môžete použiť aj inú databázu, musíte si však upraviť **persistence.xml**.
- Balík **banka** so súbormi tried:
 - **Osoba**, **Ucet** a **Prevod** sú entitné triedy do ktorých treba doplniť JPA anotácie
 - a trieda **Banka** obsahujúca dve metódy, ktoré treba implementovať.

A) Do entitných tried **Osoba**, **Ucet** a **Prevod** doplňte **JPA anotácie**, tak aby sa tieto entitné triedy namapovali na databázové tabuľky znázornené na diagrame:



Dôležité: Anotácie doplňte tak, aby z entitných tried bolo možné vygenerovať tabuľky **presne takou štruktúrou**, ako je uvedená v **diagrame**, teda:

- Názvy tabuliek, stĺpcov musia byť zachované.
- Primárny kľúč **ID** v tabuľke **PREVOD** musí byť autogenerovaný,
- Cudzie kľúče **Z_ID** a **NA_ID** sú povinné (*nie nullable*)
- V tabuľke **OSOBA** je povinný stĺpec **MENO**

Vysvetlenie k významu tabuliek a stĺpcov:

- Tabuľka **OSOBA** obsahuje informácie osobách, zákazníkoch banky: **MENO**, **ADRESU** a **ID** - primárny kľúč.
- Tabuľka **UCET** obsahuje informácie o účtoch: číslo účtu **ID** je zároveň primárny kľúč, **MAJITEL** je osoba, ktorej účet patrí a **STAV** udáva aktuálnu sumu peňazí na účte.
- Tabuľka **PREVOD** obsahuje informácie o prevodoch peňazí medzi účtami:
 - **SUMA** je prevádzaná suma,
 - **Z_ID** číslo účtu, z ktorého sa suma prevádza
 - **NA_ID** číslo účtu, na ktorý sa suma prevádza.
 - V stĺpci **ZAUCTOVANY** je čas zaúčtovania (realizácie) prevodu.
Ak prevod ešte nebol zaúčtovaný nie je v ňom žiadna hodnota.
 - Kľúč **ID** v tejto tabuľke má byť **autogenerovaný**.

B) Súbor **Banka.java** obsahuje deklarácie dvoch metód:

```
public static long vytvorPrevod( EntityManager em, double suma,  
                                long z_id, long na_id ) throws Exception  
public static int realizujPrevody( EntityManager em)
```

Implementujte metódy podľa **špecifikácie**, ktorú **nájdete v zdrojovom súbore Banka.java**, v komentári k metódam.

Poznámky k implementácii:

Odporúčam postupovať tak, že najprv dokončíte implementáciu entitných tried a z nich si necháte vygenerovať tabuľky. Nie je potrebné generovať JPAControler a ani sa **neodporúča** jeho použitie.

Informácia k JPA-mapovaniu dátových typov:

- BIGINT odpovedá v JPA číselným typom **long**, **Long**
- TIMESTAMP odpovedá dátumovému typu **Date**

*Hint: aktuálny čas získate pomocou **new Date()**;*

Pripomínam tiež, že použité mená typov BIGINT a DOUBLE sú abstraktné, t.j. v konkrétnej databáze sa môžu volať trochu odlišne. Napr. v databáze postgresql BIGINT odpovedá typu int8 a DOUBLE typy float8.

Pre vlastné testovanie si môžete implementovať program (napr. main metódu v triede banka), nie je to však súčasťou zadania, nebude sa testovať ani hodnotiť.

Pokyny pre odovzdanie

Odovzdáva sa iba adresár balíka **banka** zazipovaný. Pre zipovanie použite formát **.zip**, súbor nazvite **banka.zip**.

Implementačná úloha 2.

restapp je maven projekt pre vývoj webovej aplikácie s REST servisom. Projekt obsahuje:

- súbor **pom.xml** so všetkými závislosťami potrebnými pre vývoj servisu. Nie je ho potrebné dopĺňať.
- balík **rest**. **V tomto balíku budete vytvárať všetky triedy** (triedu implementujúcu servisu aj dátové triedu)

V balíku rest sa už nachádza konfiguračná trieda REST aplikácie JAXRSConfiguration. Netreba ju meniť.

Špecifikácia servisu

Servis bude sprístupňovať informácie o zmluvách cestovného poistenia a v nich poistených osobách.

Informácie o poistnej zmluve predstavujú resource, ktorého stav reprezentovaný vo formáte XML, má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<zmluva>
  <idZmluvy>Z123</idZmluvy>
  <vlastnik>
    <meno>Mrkva</meno>
    <rocnik>2001</rocnik>
    <adresa>Hlavna 22</adresa>
  </vlastnik>
  <pocet >2</pocet>
</zmluva>
```

Vysvetlenie

- element **idZmluvy** obsahuje jednoznačný identifikátor zmluvy.
- element **vlastnik** je podelement obsahujúcu údaje o majiteľovi poistky (meno, rocnik a adresu)
- element **pocet** udáva aktuálny počet osôb poistených v zmluve.
Ak zmluva ešte nemá poistené osoby element obsahuje **0**.
Pri pridaní alebo odobratí poistenej osoby sa počet poistencov aktualizuje.

Informácie o poistených osobách:

Každá zmluva má svoj zoznam poistených osôb. XML dokument s informáciami o zmluve však obsahuje len počet poistených osôb, **bližšie informácie o poistených osobách neobsahuje**.

Každá v zmluvou poistená osoba bude prístupná ako samostatný subresource zmluvy.

Koreňový resource servisu má URL **poistenie**

Resource s URL **poistenie/{id}**, reprezentuje poistnú zmluvu s identifikátorom **id** (pozri element idZmluvy).

Subresource s URL **poistenie/{id}/{no}**, reprezentuje osobu poistenú v zmluve. Reťazec **no** je poradové číslo v zozname zmluvou poistených osôb. Poradové čísla idú za sebou od **1** až po **pocet**.

Inicializácia servisu:

Servis inicializujte tak, že po štarte (deploymente) bude už vytvorená jedna zmluva s nasledujúcimi údajmi:

idZmluvy **Z123**,
majiteľ: meno **Mrkva**, rocnik **2001**, adresa **Hlavna 22**
nebude mať **žiadne poistené osoby**, teda pocet bude **0**.

Prístupové metódy:

Pre koreňový resource **poistenie**

- **POST** pre MediaType.APPLICATION_XML. Služí pre vytvorenie nového resoursu s informáciami o poistnej zmluve. Akceptuje XML dokument podľa hore uvedenej špecifikácie.
Metóda POST nič nevracia.

Pre subresource zmluvy **poistenie/{id}**

- **GET** pre MediaType.APPLICATION_XML. Vyhľadá zmluvu podľa identifikátora zadaného v URL a vráti informácie ako XML dokument s horeuvedenou štruktúrou.
Ak zmluva neexistuje, malo by sa vrátiť HTTP 204 alebo 404. *Pomôcka: stačí vrátiť jednoducho null.*
- **POST** pre MediaType.APPLICATION_XML. Služí na pridanie poistenej osoby do zmluvy.
Akceptuje XML dokument so štruktúrou:

```
<osoba>
  <meno>Mrkvicka</meno>
  <rocnik>20102010</rocnik>
  <adresa>Polna 1</adresa>
</osoba>
```

Vyhľadá zmluvu podľa jej id a pridá osobu do zoznamu poistených osôb v zmluve.

Návratová hodnota - MediaType.TEXT_PLAIN. Metóda POST vráti **poradové číslo osoby** v zozname.

Pripomínam: Číslovanie **začína od 1** (teda ak vkladáte na koniec zoznamu poradové číslo bude rovné dĺžke zoznamu)

Ak zmluva neexistuje, metóda vráti jednoducho **null**.

Pre subresource poistenej osoby **poistenie/{id}/{no}**

- **GET** pre MediaType.APPLICATION_XML. Vráti XML dokument poistenej osoby s daným poradovým číslom. Ak zmluva alebo poistená osoba neexistuje, malo by sa vrátiť HTTP 204 alebo 404. *Pomôcka: stačí vrátiť jednoducho **null**.*
- **DELETE** Odstráni položku zo zoznamu poistených osôb. Metóda neočakáva ani nevracia žiadne dáta. *Poznámka: Po odstránení osoby zo zoznamu poradové čísla ostatných poistených osôb v zozname môžu ale nemusia zmeniť. Nebude sa to overovať unit-testami.*

Pokyny pre implementáciu

Informácie o zmluvách (stav zdrojov) budú udržiavané len v pamäti servisu. Vytvorte si pre to vhodné dátové štruktúry a použite anotáciu **@javax.inject.Singleton**. Nezabudnite tiež na anotáciu **@XmlRootElement**

Poznámka. Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. (poistených osôb môže byť viac ale nemusí byť žiadna. Poradie elementov nie je dôležité.)

Pokyny pre odovzdanie

Odovzdáva sa iba adresár balíka **rest** zazipovaný. Pre zipovanie použite formát **.zip**, súbor nazvite **rest.zip**.